

1.5 RViz 2 可视化工具入门

所属阶段： 第一阶段 · ROS 2 入门与环境搭建

讲次数量： 4 节

预计时长： ~25 分钟

前置依赖： 1.3 开发环境搭建、1.4 认识讲师与课程路线图

学习目标

完成本节后，学员将能够：

- 描述 RViz 2 的界面布局和各面板的功能
 - 熟练使用 Display、View 和 Tool 三大核心面板
 - 添加和配置常用 Display (TF、RobotModel、LaserScan、Image、Map 等)
 - 保存和加载 RViz 配置文件 (.rviz)
 - 使用 RViz 2 可视化 TurtleBot3 的传感器数据和机器人模型
-

讲次 1: RViz 2 界面布局与核心面板

知识点

RViz 2 (Robot Visualization 2) 是 ROS 2 的官方 3D 可视化工具。它不是物理仿真器，也不会替你运行 SLAM 或导航算法；它订阅数据、查询 TF 坐标变换，并通过插件进行必要的转换和渲染。在整个课程中，RViz 2 将是调试机器人状态的重要窗口。

启动 RViz 2

```
# 确保已 source ROS 2 环境
source /opt/ros/jazzy/setup.bash

# 启动 RViz 2 (空白配置)
ros2 run rviz2 rviz2
```

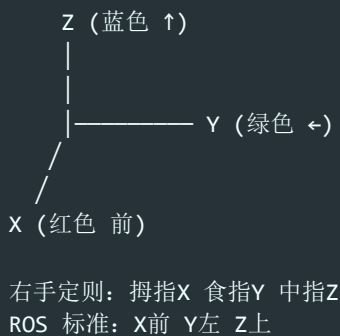
首次启动会看到一个空的 3D 视口和深灰色背景。这是正常的——RViz 2 需要手动添加 Display 才能显示内容。

界面布局总览

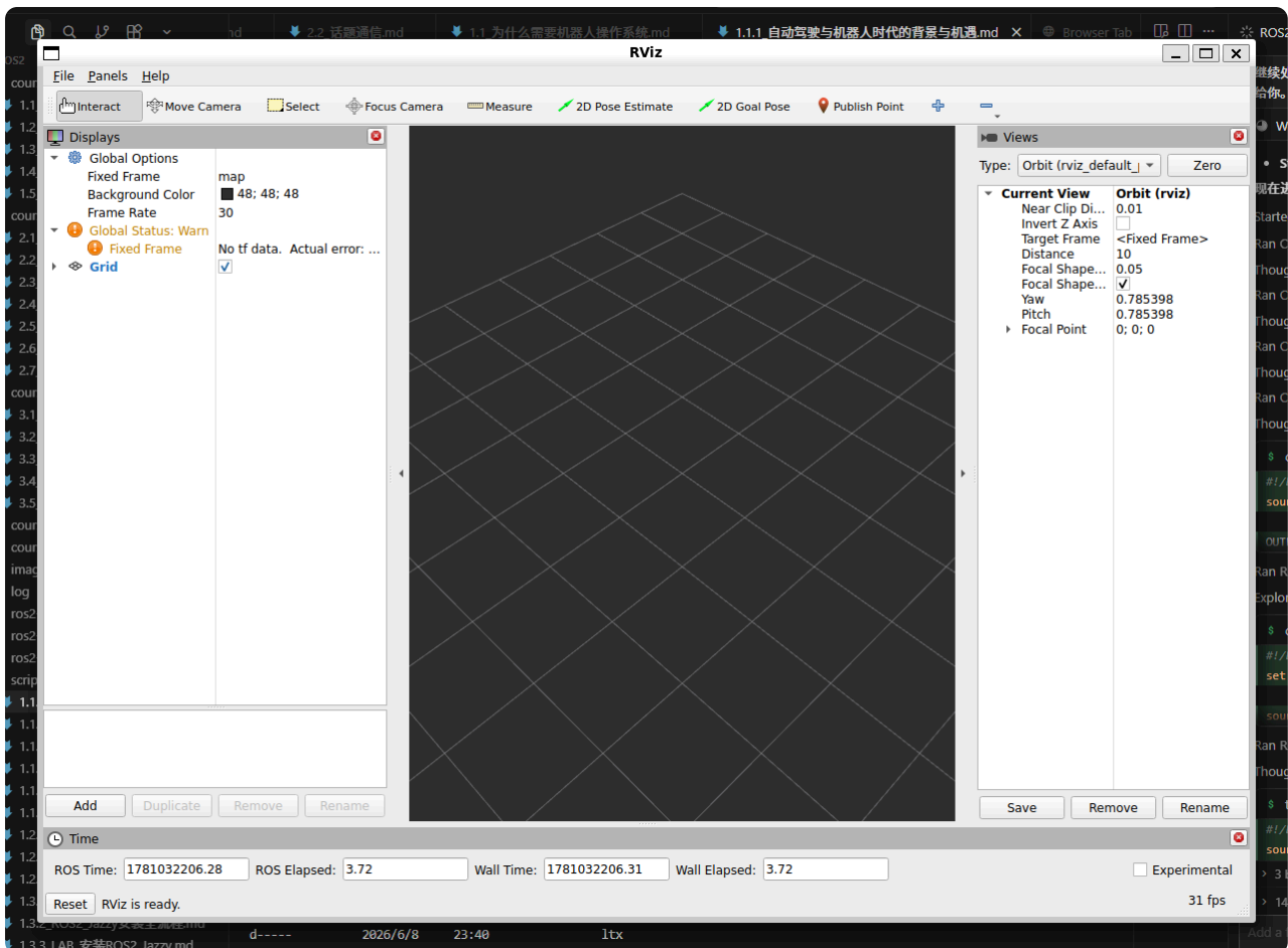
RViz 2 界面四大区域



3D 视口中 TF 坐标轴遵循 ROS 标准颜色约定（右手定则）：



右手定则：拇指X 食指Y 中指Z
ROS 标准：X前 Y左 Z上



左侧面板：Displays

Displays 是核心配置区：每个 Display 订阅一种 ROS 2 话题并渲染到 3D 视口，可独立启用/禁用。面板底部按钮：

按钮	功能
Add	添加新的 Display
Remove	删除选中的 Display
Rename	重命名 Display
↑ / ↓	调整 Display 渲染顺序

每个 Display 条目左侧的勾选框控制是否显示。红色错误图标表示配置有问题（如话题不存在）。

中心区域：3D 视口

鼠标操作取决于当前 View Controller。默认 Orbit 视角下通常使用左键旋转、中键平移、右键拖动或滚轮缩放；若操作不一致，查看 Views 面板中的当前视角类型。

右侧面板：Views

Views 面板控制**观察视角**：

参数	说明
Type	视角类型（Orbit / FPS / Third Person Follower 等）
Current View	当前视角名称
Target Frame	视角跟随的坐标系（通常 <code><Fixed Frame></code> ）
Distance	相机到焦点的距离
Focal Point	焦点坐标 (X, Y, Z)
Yaw / Pitch	水平角 / 俯仰角

常用视角：**Orbit**（默认，观察静态场景）、**FPS**（机器人视角）、**Third Person Follower**（跟踪移动）、**TopDownOrtho**（俯看地图）。点击 Zero 重置视角。

顶部工具栏：Tools

工具栏提供交互操作工具：

工具	快捷键	功能
Interact	—	默认模式，与 3D 视口交互
Move Camera	—	自由移动相机（与鼠标操作相同）
Select	—	选择 3D 对象查看属性
Focus Camera	—	将相机焦点对准选中对象
Measure	—	测量 3D 空间中两点距离
2D Pose Estimate	—	发布初始位姿估计（用于 AMCL 定位）
2D Goal Pose	—	发布导航目标位姿（用于 Nav2 导航）
Publish Point	—	在 3D 空间中点击发布一个点

导航相关：**2D Pose Estimate** 发布初始位姿到 `/initialpose`（AMCL 定位）；**2D Goal Pose** 发布目标到 `/goal_pose`（Nav2 导航）。第五阶段会频繁使用。

菜单栏

菜单	常用功能
File → Save Config As	保存当前配置为 .rviz 文件
File → Open Config	加载 .rviz 配置文件
Panels	显示/隐藏各面板（Displays、Views、Selection、Tool Properties 等）
Help	关于信息

Fixed Frame 概念

RViz 2 中最重要的全局设置是 **Fixed Frame**（固定坐标系），显示在 Displays 面板顶部：

- Fixed Frame 是所有 Display 数据变换到的目标坐标系
- 通常设置为 `map`（全局地图）或 `odom`（里程计）或 `base_link`（机器人本体）
- 如果 Fixed Frame 设置错误，所有 Display 可能显示为空白或报错
- 当没有数据时，先从 Display 的错误信息确认消息 `frame_id`，再检查该坐标系到 Fixed Frame 的 TF 是否存在；不要靠反复猜坐标系掩盖 TF 缺失

```
# 查看指定变换是否持续可用
ros2 run tf2_ros tf2_echo odom base_link

# 生成包含 /tf 和 /tf_static 的坐标树（需要 tf2_tools）
ros2 run tf2_tools view_frames
```

讲次 2：添加和配置常用 Display

知识点

Display 是 RViz 2 的基本组成单元。每个 Display 负责将一种 ROS 2 消息类型可视化。本节介绍最常用的 Display 类型及其配置方法。

添加 Display 的通用流程

1. 点击 Displays 面板底部的 **Add** 按钮
2. 在弹出窗口中选择 **By display type** 标签
3. 从列表中选择需要的 Display 类型
4. 点击 **OK**
5. 在左侧面板中展开新 Display，配置参数（主要是 Topic）

常用 Display 详解

1. Grid（网格）

最基础的 Display，显示地面参考网格。

参数	推荐值	说明
Reference Frame	<Fixed Frame>	跟随固定坐标系
Plane Cell Count	10	网格数量
Cell Size	1.0	每格 1 米
Color	160; 160; 164	浅灰色

通常作为第一个添加的 Display，提供空间尺度参考。

2. TF（坐标变换）

可视化 ROS 2 TF 树中的所有坐标系：

参数	说明
Show Names	显示坐标系名称标签
Show Axes	显示 XYZ 轴（红X/绿Y/蓝Z）
Show Arrows	显示坐标系间的变换箭头
Marker Scale	坐标轴长度（默认 1.0 米）
Frame Timeout	超时未更新的坐标系变红

```
TF 树示例（移动机器人）：
map → odom → base_link → laser_link
                        → camera_link
                        → imu_link
```

无需配置话题——TF Display 自动订阅 `/tf` 和 `/tf_static` 话题。

3. RobotModel (机器人模型)

显示 URDF 定义的机器人 3D 模型：

参数	说明
Description Source	模型来源（通常选 Topic）
Description Topic	Topic 模式下通常订阅 <code>/robot_description</code> ；具体来源以当前 RViz 版本和启动配置为准
Visual Enabled	显示视觉模型（外观）
Collision Enabled	显示碰撞模型（通常关闭）

需要配合 `robot_state_publisher` 使用：它读取 URDF，订阅 `/joint_states`，再发布 `/tf` 与 `/tf_static`；关节状态通常由驱动、仿真器或 `joint_state_publisher` 发布，而不是由 `robot_state_publisher` 产生。

4. LaserScan (激光雷达扫描)

显示 2D 激光雷达的扫描数据：

参数	推荐值	说明
Topic	<code>/scan</code>	激光雷达话题
Size (m)	0.05	每个扫描点的大小
Style	Flat Squares	显示样式
Color Transformer	Intensity / FlatColor	颜色映射方式
Decay Time	0	0 = 只显示最新一帧

数据来自 `sensor_msgs/LaserScan` 消息，显示为扇形点云。

5. Image (相机图像)

显示相机发布的图像：

参数	说明
Topic	图像话题（如 <code>/camera/image_raw</code> ）
Transport	传输方式（raw / compressed）
Normalize Range	是否归一化像素值

Image Display 通常作为 RViz 中的二维面板显示图像，并不会自动把相机画面贴到 3D 世界中的平面。若要在 3D 场景中投影，需要 Camera Display、相机标定信息或自定义可视化方案。

6. Map (占用栅格地图)

显示 SLAM 或导航地图：

参数	推荐值	说明
Topic	/map	地图话题
Color Scheme	map	标准配色（白=自由，黑=障碍，灰=未知）
Alpha	0.7	透明度
Draw Behind	true	地图渲染在其他 Display 后面

数据来自 `nav_msgs/OccupancyGrid` 消息。

其他常用 Display：**MarkerArray**、**PointCloud2**、**Path**——后续阶段逐步使用。

Display 状态指示

状态	图标颜色	含义
正常	绿色勾	话题有数据，正常显示
等待	黄色	话题存在但暂无数据
错误	红色	话题不存在或配置错误

当 Display 显示红色错误时，检查：

```
# 1. 确认话题是否存在
ros2 topic list | grep <topic_name>

# 2. 确认话题有数据发布
ros2 topic hz /scan

# 3. 确认消息类型匹配
ros2 topic info /scan
```



讲次 3：保存和加载 RViz 配置文件

知识点

每次手动配置 Display 很繁琐。RViz 2 支持将当前配置保存为 `.rviz` 文件，下次直接加载即可恢复所有 Display 设置、视角位置 and 工具配置。

保存配置

方法一：通过菜单

1. File → Save Config As...
2. 选择保存路径，输入文件名（如 `navigation.rviz`）
3. 点击 Save

加载配置

方法一：File → Open Config

方法二：命令行加载（推荐）

```
# 启动时直接加载配置
ros2 run rviz2 rviz2 -d ~/ros2_ws/navigation.rviz

# Launch 文件中: arguments=['-d', rviz_config_path]
```

配置文件管理最佳实践

将 `.rviz` 文件放在包的 `config/` 目录或工作空间根目录。本课程预设见仓库 `stage1_code/config/turtlebot3_basic.rviz`（1.5 LAB 可直接加载）。

多环境配置策略

配置文件	用途	包含的 Display
<code>default.rviz</code>	通用调试	Grid + TF + RobotModel
<code>slam.rviz</code>	SLAM 建图	+ LaserScan + Map
<code>nav.rviz</code>	自主导航	+ Path + MarkerArray + 2D Goal
<code>perception.rviz</code>	感知调试	+ Image + PointCloud2

默认配置路径：`/opt/ros/jazzy/share/rviz_default_plugins/rviz_default.rviz`（系统）、`~/rviz2/default.rviz`（用户）。

讲次 4: 【LAB】用 RViz 2 可视化 TurtleBot3 数据

知识点

本实验将安装 TurtleBot3 相关包，启动仿真机器人，并在 RViz 2 中可视化其激光雷达数据和机器人模型。

前置条件

- 已完成 1.3 节 ROS 2 Jazzy 环境搭建
- RViz 2 可正常启动

实验步骤

Step 1: 安装 TurtleBot3 包和 Gazebo 桥接

```
sudo apt update
sudo apt install -y \
  ros-jazzy-turtlebot3 \
  ros-jazzy-turtlebot3-msgs \
  ros-jazzy-turtlebot3-simulations \
  ros-jazzy-ros-gz
```

重要： Jazzy 的当前 TurtleBot3 仿真使用 Gazebo Harmonic 与 `ros_gz`。元包可能已经间接安装桥接组件，这里显式安装是为了让实验依赖清楚、可复现。

验证安装：

```
ros2 pkg list | grep turtlebot3
# 预期输出包含：
# turtlebot3_gazebo
# turtlebot3_description
# turtlebot3_teleop
```

Step 2: 设置 TurtleBot3 型号环境变量

```
# 写入 .bashrc (永久生效; grep 先查重, 避免重复执行时追加多行)
grep -Fqx "export TURTLEBOT3_MODEL=burger" ~/.bashrc || echo "export TURTLEBOT3_MODEL=burger" >>
source ~/.bashrc

# 验证
echo $TURTLEBOT3_MODEL
# 预期: burger
```

TurtleBot3 有三种型号: `burger` (小型)、`waffle` (中型)、`waffle_pi` (带相机)。



Step 3: 启动 TurtleBot3 仿真 (Gazebo)

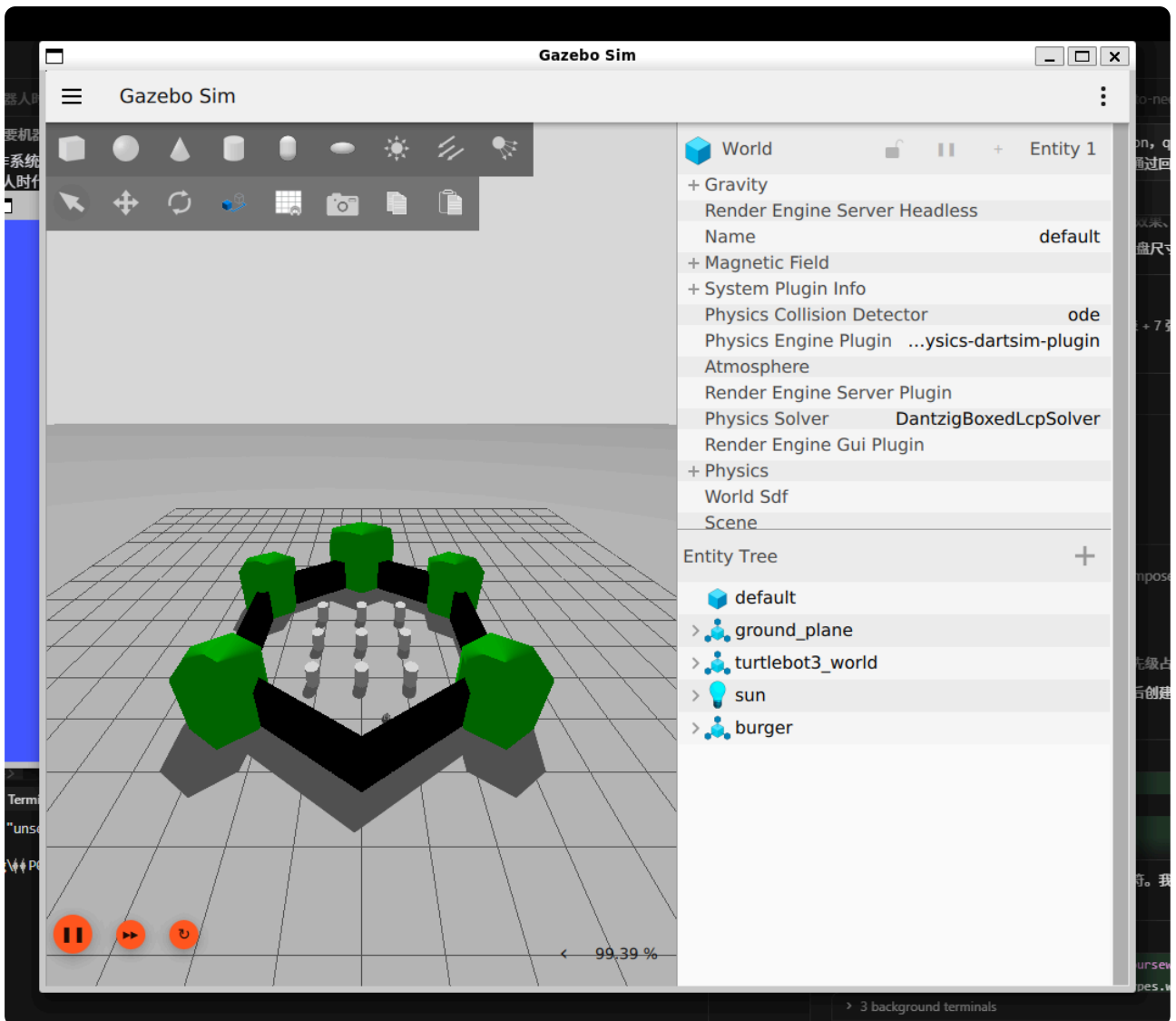
```
# 终端 1: 启动 Gazebo 仿真环境
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

预期输出 (Jazzy 使用 Gazebo Harmonic, 日志格式与 Gazebo Classic 不同) :

```
[INFO] [robot_state_publisher]: Publishing robot description
[INFO] [ros_gz_sim]: Waiting for world [turtlebot3_world] ...
[INFO] [ros_gz_bridge]: Creating bridge for topic [/scan] ...
[INFO] [create-3]: Requesting entity spawn ...
[INFO] [create-3]: Entity successfully spawned
```

注意: Jazzy 使用 Gazebo Harmonic (`ros_gz`), 日志格式与旧版 Gazebo Classic 不同。如果看到 `ros_gz_sim` 和 `create` 相关日志, 这是正常的 Harmonic 行为。

Gazebo 窗口弹出, 显示 TurtleBot3 机器人在虚拟世界中。首次启动可能需要下载模型资源, 耐心等待 1-2 分钟。



验证仿真正常：`ros2 node list` 应显示多个节点，`ros2 topic list` 应包含 `/scan`、`/odom`、`/tf` 等话题。

常见问题排查

- 如果 Gazebo 闪退或黑屏：确认已安装 `ros-jazzy-turtlebot3-gazebo` 和 `ros-jazzy-ros-gz`
- 如果 RViz 中看不到机器人模型：检查 Fixed Frame 是否设为 `odom` 而非 `map`
- 如果 Gazebo 模型加载缓慢：首次运行需从 Gazebo Fuel 下载模型，请保持网络畅通

Step 4: 启动 RViz 2 并加载 Display

方式 A (推荐)：直接加载课程仓库预设（需已 clone 本仓库，将 `REPO` 换为实际路径）：

```
# 终端 2
source /opt/ros/jazzy/setup.bash
ros2 run rviz2 rviz2 -d REPO/stage1_code/config/turtlebot3_basic.rviz
```

预设已包含：Fixed Frame: `odom`、Grid、TF、RobotModel（`/robot_description`）、LaserScan（`/scan`）。使用 `odom` 是为了在机器人移动时看到模型相对里程计坐标系发生位移；改成 `base_link` 后机器人会保持在视口中心。

方式 B: 空白启动后按讲次 2 手动配置:

```
source /opt/ros/jazzy/setup.bash
ros2 run rviz2 rviz2
```

添加 Grid、TF (勾选 Show Names / Show Axes)、RobotModel、LaserScan (Topic: `/scan`, Size: 0.05) ; Views 面板 Distance 设为 3.0。

Step 5: 用键盘驱动机器人并观察激光数据变化

```
# 终端 3: 键盘遥控
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger
ros2 run turtlebot3_teleop teleop_keyboard
```

在 teleop 终端中按程序提示使用字母键控制: `i` 前进、`o` 后退、`j/l` 转向、`k` 停止。驱动机器人移动, 观察 RViz 2 中: - Fixed Frame 为 `odom` 时, 机器人模型在网格中移动; Fixed Frame 为 `base_link` 时, 模型保持居中而周围数据相对变化 - 激光扫描数据实时更新 - TF 坐标系跟随变化

Step 6: 保存或使用配置文件

若采用方式 A, 可跳过保存步骤。若采用方式 B 手动配置, 在 RViz 2 中 File → Save Config As... 保存为 `~/ros2_ws/turtlebot3.rviz`。

也可直接复制仓库预设:

```
cp REPO/stage1_code/config/turtlebot3_basic.rviz ~/ros2_ws/turtlebot3.rviz
```

下次加载:

```
ros2 run rviz2 rviz2 -d ~/ros2_ws/turtlebot3.rviz
```

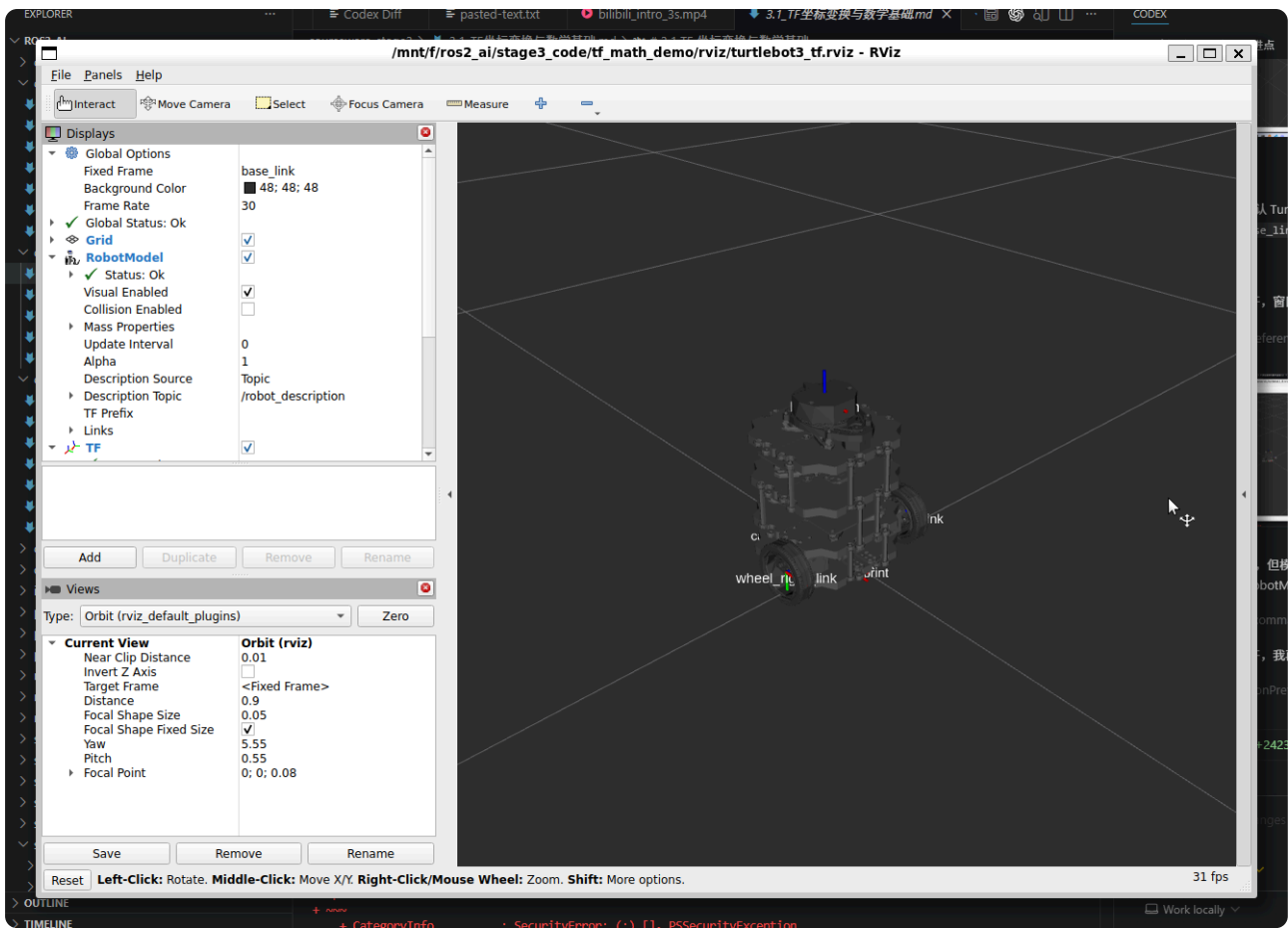
Step 7: 验证话题 (可选)

```
ros2 topic list    # 应含 /scan, /odom, /cmd_vel, /tf
ros2 topic hz /scan # 预期 ~5-10 Hz
```

预期最终效果

RViz 2 中应显示:

- 灰色地面网格
- TurtleBot3 蓝色圆柱形机器人模型
- 红色/彩色激光扫描点在机器人前方形成扇形
- 多个 TF 坐标系 (base_link 为中心, 带 XYZ 轴)
- 移动机器人时所有元素实时更新



常见错误与解决方案

错误信息 / 现象	原因	解决方案
RobotModel 红色错误	robot_state_publisher 未运行	确认 Gazebo launch 已正常启动
LaserScan 无数据	话题名不匹配	检查 <code>ros2 topic list</code> , 可能需改为 <code>/scan</code>
所有 Display 空白	Fixed Frame 与消息坐标系之间没有 TF	用 <code>tf2_echo / view_frames</code> 查明缺失变换, 再选择 <code>odom</code> 或 <code>base_link</code>
Gazebo 启动失败	缺少 Gazebo 包	<code>sudo apt install ros-jazzy-ros-gz</code>
<code>TURTLEBOT3_MODEL</code> 未设置	环境变量缺失	<code>export TURTLEBOT3_MODEL=burger</code>
RViz 中机器人不移动	未启动 teleop	启动 <code>turtlebot3_teleop teleop_keyboard</code>
Gazebo 窗口黑屏	显卡/渲染问题	虚拟机启用 3D 加速; 尝试 <code>export LIBGL_ALWAYS_SOFTWARE=1</code>
<code>package 'turtlebot3_gazebo' not found</code>	未安装仿真包	<code>sudo apt install ros-jazzy-turtlebot3-simulations</code>

清理

实验结束后, 在各终端中按 `Ctrl+C` 停止节点。如果确认没有其他 Gazebo 会话且该 launch 未正常退出, 可使用:

```
pkill -TERM -f 'gz sim'
```



本节总结

关键命令速查表

命令	说明
<code>ros2 run rviz2 rviz2</code>	启动 RViz 2（空白配置）
<code>ros2 run rviz2 rviz2 -d config.rviz</code>	加载配置文件启动
<code>export TURTLEBOT3_MODEL=burger</code>	设置 TurtleBot3 型号
<code>ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py</code>	启动 TurtleBot3 Gazebo 仿真
<code>ros2 run turtlebot3_teleop teleop_keyboard</code>	键盘遥控 TurtleBot3
<code>ros2 topic hz /scan</code>	查看激光雷达数据频率
<code>ros2 run tf2_tools view_frames</code>	生成 TF 树 PDF 图

常用 Display 速查

Display	消息类型	典型话题
Grid	—	—
TF	tf2_msgs/TFMessage	/tf, /tf_static
RobotModel	URDF 描述（Jazzy Topic 模式常为 std_msgs/String）+ sensor_msgs/JointState + TF	/robot_description、/joint_states、/tf
LaserScan	sensor_msgs/LaserScan	/scan
Image	sensor_msgs/Image	/camera/image_raw
Map	nav_msgs/OccupancyGrid	/map
PointCloud2	sensor_msgs/PointCloud2	/points
Path	nav_msgs/Path	/plan
MarkerArray	visualization_msgs/MarkerArray	/markers

第一阶段总结

完成第一阶段后，你已具备 ROS 2 入门所需的理论、环境与可视化调试能力。进入 **第二阶段：ROS 2 核心通信机制**，从第一个节点开始系统学习 Topic、Service、Action、Launch 和 Parameter，并完成 🌈 Turtlesim 通信实战项目。